

**YENEPOYA INSTITUTE OF ARTS,
SCIENCE,
COMMERCE AND MANAGEMENT**

Project – High Level Design

on

Behavioral Analytics for Energy Security

Student Name: Veekshith Poojary

Industry Mentor:
Mr.Sumith
Kumar Shukla

Guided
by:Divya N
Shetty

Table of Contents

- 1. Introduction**
 - 1.1 Scope of the document**
 - 1.2 Intended Audience**
 - 1.3 System overview**
- 2. System Design**
 - 2.1 Application Design**
 - 2.2 Process Flow**
 - 2.3 Information Flow**
 - 2.4 Components Design**
 - 2.5 Key Design Considerations**
 - 2.6 API Catalogue**
- 3. Data Design**
 - 3.1 Data Model**
 - 3.2 Data Access Mechanism**
 - 3.3 Data Retention Policies**
 - 3.4 Data Migration**
- 4. Interfaces**
- 5. State and Session Management**
- 6. Caching**
- 7. Non-Functional Requirements**
 - 7.1 Security Aspects**
 - 7.2 Performance Aspects**
- 8. References**

1. Introduction

1.1 Scope of the Document

This document presents the High Level Design (HLD) for the Behavioral Analytics for Energy Security platform. It describes the system architecture, component design, data flows, interface specifications, security considerations, and non-functional requirements. This document serves as the architectural blueprint for development, review, and evaluation of the system.

1.2 Intended Audience

- Software Developers and Engineers
- Project Reviewers and Evaluators
- Academic Supervisors and Industry Mentors
- Security Operations Center (SOC) Analysts
- System Architects

1.3 System Overview

Behavioral Analytics for Energy Security is a specialized cybersecurity platform designed for the Energy Sector. The system ingests user login log data, analyzes behavioral patterns, and identifies potential insider threats or compromised credentials through a dual-engine approach:

- **Rule-Based Engine:** Deterministic detection of known threat signatures using 5 security rules
- **Machine Learning Engine:** Unsupervised anomaly detection using Isolation Forest (Scikit-Learn) to identify subtle, unknown behavioral outliers

The system is built on Python and Streamlit, providing an interactive web-based dashboard for Security Operations Center (SOC) operators to upload login datasets, tune detection parameters, investigate threats visually, and export detailed anomaly reports.

2. System Design

2.1 Application Design

The application is designed as a modular, single-tier web application with no persistent backend database. All processing is performed in-memory using Pandas DataFrames. The architecture comprises the following layers:

Layer	Technology	Responsibility
Presentation Layer	Streamlit (Python)	Web UI: sidebar controls, tabbed dashboard, charts, report download
Business Logic Layer	Python Modules (src/)	Data loading, preprocessing, rule-based detection, risk scoring, ML engine
Visualization Layer	Plotly Express / Graph Objects	7 interactive dark-theme charts rendered client-side
Persistence Layer	Joblib (.pkl), CSV files	Serialized ML models in models/; exported reports in reports/
Styling Layer	CSS (assets/style.css)	Custom SOC-style dark mode theme applied to Streamlit UI

2.2 Process Flow

The end-to-end system workflow is as follows:

[Process Flow Diagram should be inserted here showing the following sequence:]

- **Step 1 – Data Ingestion:** User uploads CSV or system loads sample_data/login_data.csv
- **Step 2 – Schema Validation:** Required columns (user_id, login_time, status, location, device) validated
- **Step 3 – Preprocessing:** Null removal, timestamp parsing, string normalization
- **Step 4 – Feature Engineering:** Temporal features (hour, day, weekday, month, is_failed) derived
- **Step 5 – Baselineing:** Per-user patterns aggregated (avg login hour, total logins, known devices)
- **Step 6 – Rule-Based Detection:** 5 security rules applied; anomaly flags and reasons recorded
- **Step 7 – ML Detection (Optional):** Isolation Forest trained and scored; -1 mapped to Suspicious
- **Step 8 – Risk Consolidation:** Rule scores summed, capped at 100, categorized Low/Medium/High/Critical
- **Step 9 – Dashboard Rendering:** KPI metrics, data tables, 7 Plotly charts populated
- **Step 10 – Report Export:** Anomaly CSV saved to server or downloaded via browser

2.3 Information Flow

Data flows unidirectionally through the pipeline from ingestion to presentation:

Stage	Input	Output	Module
File Upload	CSV file (user upload or local path)	Raw Pandas DataFrame	data_loader.py
Column Validation	Raw DataFrame	Validated DataFrame or error	data_loader.py
Data Cleaning	Validated DataFrame	Cleaned DataFrame (no nulls)	preprocessing.py
Feature Engineering	Cleaned DataFrame	Feature-enriched DataFrame	preprocessing.py
User Baselineing	Feature DataFrame	User patterns DataFrame	detector.py
Rule-Based Detection	Feature + Patterns DataFrames	Anomaly-flagged DataFrame	detector.py
Risk Scoring	Flagged DataFrame	Risk-scored DataFrame with levels	detector.py
ML Detection	Feature DataFrame	ML-flagged DataFrame (Suspicious/Normal)	ml_model.py
Result Merge	Rule + ML DataFrames	Combined anomaly DataFrame	app.py

Stage	Input	Output	Module
Visualization	Combined DataFrame	7 Plotly chart figures	dashboard.py
Report Export	Anomaly-only DataFrame	CSV file (server or browser download)	utils.py

2.4 Components Design

Component	File	Type	Description
Main Application	app.py	Orchestrator	Streamlit entry point; manages session state, UI layout, pipeline execution
Data Loader	src/data_loader.py	Utility Module	Handles CSV ingestion and schema validation
Preprocessor	src/preprocessing.py	Processing Module	Data cleaning and temporal feature engineering
Rule Engine	src/detector.py	Detection Module	Deterministic 5-rule threat detection and risk scoring
ML Engine	src/ml_model.py	ML Module	Isolation Forest training, serialization, and anomaly prediction

Component	File	Type	Description
Dashboard	src/dashboard.py	Visualization Module	7 Plotly dark-theme interactive charts
Report Utils	src/utlis.py	Utility Module	CSV export to server and browser download stream
Style Sheet	assets/style.css	UI Asset	Custom SOC dark-mode CSS theme for Streamlit
Sample Data	sample_data/login_data.csv	Data Asset	Default login dataset used when no file is uploaded
ML Models Dir	models/	Storage	Auto-created directory storing isolation_forest.pkl and device_encoder.pkl
Reports Dir	reports/	Storage	Auto-created directory storing exported anomaly CSV reports

2.5 Key Design Considerations

- **Stateless Processing:** No persistent database; all state managed in Streamlit `session_state` as DataFrames, enabling simple deployment
- **Dual-Engine Detection:** Combining deterministic rule logic with unsupervised ML ensures both known signature detection and novel anomaly discovery
- **Energy Sector Focus:** Detection rules (3 AM logins, device entropy, time deviation) are tuned to the operational patterns of critical infrastructure environments
- **Explainability:** Every flagged event carries an `anomaly_reason` string, making detections interpretable for SOC analysts
- **Modularity:** Each `src/` module is independently testable and replaceable without affecting other components
- **Offline Capability:** System operates fully on-premise; no external API calls or cloud dependencies
- **Model Persistence:** Isolation Forest and LabelEncoder serialized via Joblib, enabling model reuse across sessions

2.6 API Catalogue

This system is a standalone web application and does not expose external REST or HTTP APIs. Internal function interfaces between modules are as follows:

Function Signature	Module	Returns
<code>load_file(source: str UploadedFile) -> DataFrame</code>	<code>data_loader.py</code>	Pandas DataFrame
<code>validate_columns(df: DataFrame, required: list) -> bool</code>	<code>data_loader.py</code>	True or raises ValueError

Function Signature	Module	Returns
clean_data(df: DataFrame) -> DataFrame	preprocessing.py	Cleaned DataFrame
extract_features(df: DataFrame) -> DataFrame	preprocessing.py	Feature-enriched DataFrame
calculate_user_patterns(df: DataFrame) -> DataFrame	detector.py	User baselines DataFrame
detect_rule_based(data, patterns, threshold: int) -> DataFrame	detector.py	Anomaly-flagged DataFrame
calculate_risk(df: DataFrame) -> DataFrame	detector.py	Risk-scored DataFrame
train_model(login_data: DataFrame) -> (model, encoder)	ml_model.py	IsolationForest, LabelEncoder
predict_anomalies(model, features: DataFrame) -> DataFrame	ml_model.py	DataFrame with ml_flag column
save_report(df: DataFrame, filepath: str) -> None	utils.py	None (writes CSV to disk)
export_csv(df: DataFrame) -> str	utils.py	UTF-8 encoded CSV string

3. Data Design

3.1 Data Model

The system operates on a flat login event schema. The core data model is:

Field	Type	Description	Required
user_id	String	Unique identifier for the user account	Yes
login_time	DateTime	Timestamp of the login event (parsed to Pandas datetime)	Yes
status	String	Login outcome: 'Success' or 'Failed'	Yes
location	String	Geographic location of login attempt	Yes
device	String	Device identifier used for the login	Yes
hour	Integer	Derived: hour of login_time (0-23)	Engineered
day	Integer	Derived: day of month from login_time	Engineered
weekday	Integer	Derived: day of week (0=Monday)	Engineered
month	Integer	Derived: month from login_time	Engineered

Field	Type	Description	Required
is_failed	Integer (0/1)	Derived: 1 if status is 'Failed', else 0	Engineered
is_anomaly	Boolean	Flagged: True if any rule or ML marks as anomalous	Computed
anomaly_reason	String	Computed: pipe-separated list of triggered rule reasons	Computed
risk_score	Integer (0-100)	Computed: cumulative risk points from triggered rules	Computed
risk_level	String	Computed: Low / Medium / High / Critical category	Computed
ml_flag	String	Computed: 'Suspicious' or 'Normal' from Isolation Forest	Computed

3.2 Data Access Mechanism

- All data is loaded into memory as Pandas DataFrames; no SQL or NoSQL database is used
- CSV files are read via `pd.read_csv()` with UTF-8 encoding
- ML models are serialized to disk using `joblib.dump()` and loaded via `joblib.load()`
- Exported reports are written to the `reports/` directory using `DataFrame.to_csv()`
- Streamlit `session_state` dictionary is used to persist DataFrames between user interactions within a session

3.3 Data Retention Policies

- **In-Memory Data:** All processed DataFrames exist only for the duration of the Streamlit session; cleared on page reload or session end
- **Exported Reports:** Anomaly CSV reports saved to the local reports/ directory are retained indefinitely until manually deleted
- **ML Models:** Serialized .pkl model files in models/ persist on disk across sessions; overwritten each time the analysis pipeline is re-run
- **No user data** is transmitted to external servers or stored in any cloud storage

3.4 Data Migration

This system does not require data migration as it operates on user-supplied CSV files without a persistent database. If a new version of the system introduces schema changes, users should ensure their CSV files conform to the updated required column specification: user_id, login_time, status, location, device.

4. Interfaces

Interface	Type	Description
CSV File Upload	User Input Interface	Streamlit file uploader widget in sidebar; accepts .csv files; falls back to sample_data/login_data.csv if empty
Threshold Slider	User Input Interface	Streamlit slider (range 1-20, default 5) to configure failed attempts detection threshold
ML Toggle	User Input Interface	Streamlit checkbox to enable or disable the Isolation Forest ML detection pipeline
Run Analysis Button	Trigger Interface	Streamlit button that initiates the full detection and visualization pipeline
Overview Tab	Output Interface	Displays KPI metric cards: Total Users, Total Logins, Anomalies Found, High Risk Users; dataset preview
Behavioral Analysis Tab	Output Interface	Filterable data table of flagged events; bar chart of Risk Score distribution
Dashboard Tab	Output Interface	7 interactive Plotly charts in SOC dark theme for visual threat investigation
Reports Tab	Output Interface	Anomaly-only event table; Save Report button (server-side); Download CSV button (browser-side)
About Tab	Documentation Interface	Static project documentation: purpose, detection logic, usage guide

Interface	Type	Description
reports/ directory	File System Interface	Local directory where server-side anomaly CSV reports are persisted
models/ directory	File System Interface	Local directory where Joblib-serialized .pkl model files are stored and retrieved

5. State and Session Management

Streamlit re-executes the entire Python script on every user interaction. To prevent expensive recomputation, processed DataFrames and analysis results are stored in Streamlit's `st.session_state` dictionary:

Session State Key	Type	Description
processed_df	Pandas DataFrame	Cleaned and feature-engineered login DataFrame
anomaly_df	Pandas DataFrame	Combined rule-based and ML anomaly results
user_patterns	Pandas DataFrame	Per-user baseline statistics
ml_model	IsolationForest	Trained Isolation Forest model object
ml_encoder	LabelEncoder	Fitted label encoder for device field

Session state is initialized at application start and cleared on browser page reload. No cross-session state persistence exists beyond the serialized model .pkl files on disk.

6. Caching

- Streamlit's `@st.cache_data` decorator can be applied to `load_file()` and `clean_data()` functions to cache results per unique input file hash, avoiding redundant reprocessing on repeated uploads of the same file
- Trained Isolation Forest and LabelEncoder objects are serialized to `models/isolation_forest.pkl` and `models/device_encoder.pkl` via Joblib, acting as a persistent model cache across sessions
- Plotly chart objects are generated once per analysis run and stored in `session_state` to prevent re-rendering on tab switches
- No external caching systems (Redis, Memcached) are required given the in-memory, single-user design of the application

7. Non-Functional Requirements

7.1 Security Aspects

Security Concern	Design Decision
Data Privacy	All data processing occurs locally on the server; no user data is sent to external services or APIs
File Validation	Uploaded files validated for schema compliance before processing; malformed files are rejected with error messages
No Authentication Layer	The system is designed for internal SOC use; authentication is assumed to be handled at the infrastructure/network level
Model Security	Serialized .pkl files are stored locally and not exposed over any network endpoint

Security Concern	Design Decision
Report Access Control	Exported reports stored in reports/ directory; access controlled by the operating system's file permission model
No SQL Injection Risk	No database queries exist; all data manipulation uses Pandas DataFrame operations on in-memory objects

7.2 Performance Aspects

Performance Dimension	Design Decision
Processing Speed	In-memory Pandas operations ensure fast dataset processing; no I/O bottlenecks from database queries
ML Training Time	Isolation Forest training is $O(n \log n)$; suitable for typical login log sizes (thousands to tens of thousands of records)
Model Reuse	Trained models serialized to .pkl files; reloaded on subsequent runs to avoid retraining overhead
Visualization Performance	Plotly renders charts client-side in JavaScript; server only computes data, not rendering
Scalability	System is designed for single-session use; for multi-user SOC deployment, containerization via Docker is recommended
Memory Footprint	DataFrames stored in session_state; large datasets (>1M rows) may require chunked processing or server-side filtering

8. References

- **Streamlit Documentation** — <https://docs.streamlit.io>
- **Pandas Documentation** — <https://pandas.pydata.org/docs>
- **NumPy Documentation** — <https://numpy.org/doc>
- **Scikit-Learn Documentation (IsolationForest, PCA, LabelEncoder)** — <https://scikit-learn.org/stable>
- **Plotly Python Documentation** — <https://plotly.com/python>
- **Joblib Documentation** — <https://joblib.readthedocs.io>
- **MITRE ATT&CK Framework for ICS (Energy Sector Threat Reference)** — <https://attack.mitre.org/matrices/ics>
- **Python Software Foundation** — <https://docs.python.org>